

The Software Process: Standards, Models & Life Cycles

Topic 2 — Software Processes & Standards (part 1) · **LO1** (Learning Outcome 1)

You learn the standard ways in which the industry organises software work, argue for one of them, and choose — with numbers — the process your own product will follow.



Cyber.SoHo Educational Hub
From the Industry to the
Classroom



The hour, minute by minute

Time	What
0:00–0:11	Lesson 1 — What a software process is
0:11–0:15	 Visualize-it — draw your product's process flow
0:15–0:24	Lesson 2 — Assembling a life cycle from standard activity groups
0:24–0:28	 Flash-cards — sort the activity groups
0:28–0:38	Lesson 3 — The prescriptive models
0:38–0:42	Lesson 4 — Unified Process, components, formal methods
0:42–0:46	 Fun-quiz — which model would you pick?
0:46–0:56	Lesson 5 — Choosing a model, and the first numbers
0:56–1:00	 Zen-break, then  Try-it — effort and time for 5 KLOC (thousands of Lines Of Code)
1:00–1:10	Break — ten real minutes
1:10–2:50	Studio — Model Fair, decision record, life-cycle map, numbers
2:50–3:00	Wrap-up — Block 3 brief and exit ticket
at home	Home Sprint (120') — finish Block 3, read, practise



Today's question

- In what order will we do the work?
- How many times will we repeat each part?
- How will we know a part is finished?
- The answer is a **software process** — first of two lessons on **LO1** (Learning Outcome 1)

ON THE BOARD ALL HOUR

PROCESS = what · in which order · how often · done when?

The answer to these four questions, written down for one project, is its **software process**.



Process, activity, action, task

- **software process:** activities, actions and tasks performed to create a work product
- **activity** — broad objective, every project (communication)
- **action** — set of tasks → major work product (architectural design)
- **task** — small, well-defined, tangible outcome (run one unit test)
- **work product** — what the process produces and keeps

KEY TERM

software process

A collection of activities, actions and tasks performed when a work product is to be created. The team adapts it: it selects the actions and tasks that suit the project, its people and its deadline.



The five framework activities

- **Communication** — objectives and requirements
- **Planning** — tasks, risks, resources, schedule
- **Modeling** — requirements and design models
- **Construction** — code + test
- **Deployment** — deliver, evaluate, feedback
- Same five in every project; detail, order and repetition change

KEY TERM

process framework

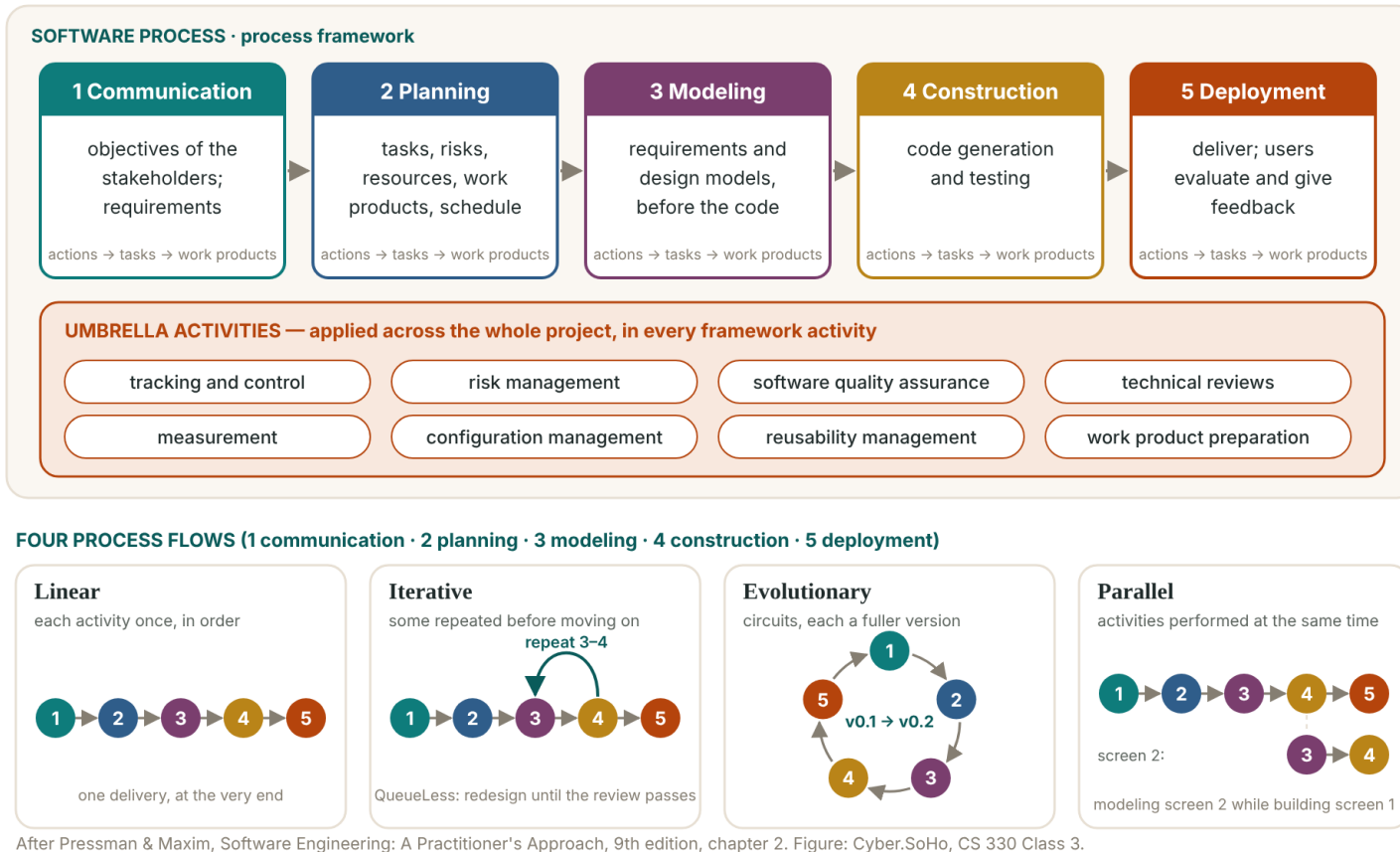
The foundation of a complete software process: a small set of framework activities that apply to every software project, together with the umbrella activities that apply across them.



Figure 1 — The generic process framework

The generic process framework

Five framework activities, eight umbrella activities, and four ways to organise them in time (process flows)



The five framework activities, the eight umbrella activities that run across them, and the four process flows. Use it for the Visualize-it break: your drawing should resemble one of the four small panels.



Umbrella activities

- Applied across the whole project, not in one stage
- tracking and control · risk management · **SQA** (Software Quality Assurance)
- technical reviews · measurement · **SCM** (Software Configuration Management)
- reusability management · work product preparation
- You already do four: stand-up, harms table, review, one Git tag per Block

KEY TERM

umbrella activities

Activities applied throughout the whole project, across all the framework activities, to manage and control progress, quality, change and risk.



Process flows

- **process flow**: how activities are organised in sequence and time
- **linear** — each activity once, in order
- **iterative** — repeat some activities before moving on
- **evolutionary** — circuits; each one a more complete version
- **parallel** — activities at the same time
- This course: Sprint 0 → 1 → 2 = evolutionary

KEY TERM

process flow

The way the framework activities, and the actions and tasks inside them, are organised in sequence and in time.



Process patterns

- **process pattern**: a recurring process problem + its context + a proven solution
- **stage pattern** — one framework activity
- **task pattern** — one action or task
- **phase pattern** — a sequence of activities (spiral, prototyping)
- Template: name, forces, type, contexts, problem, solution, related, known uses

KEY TERM

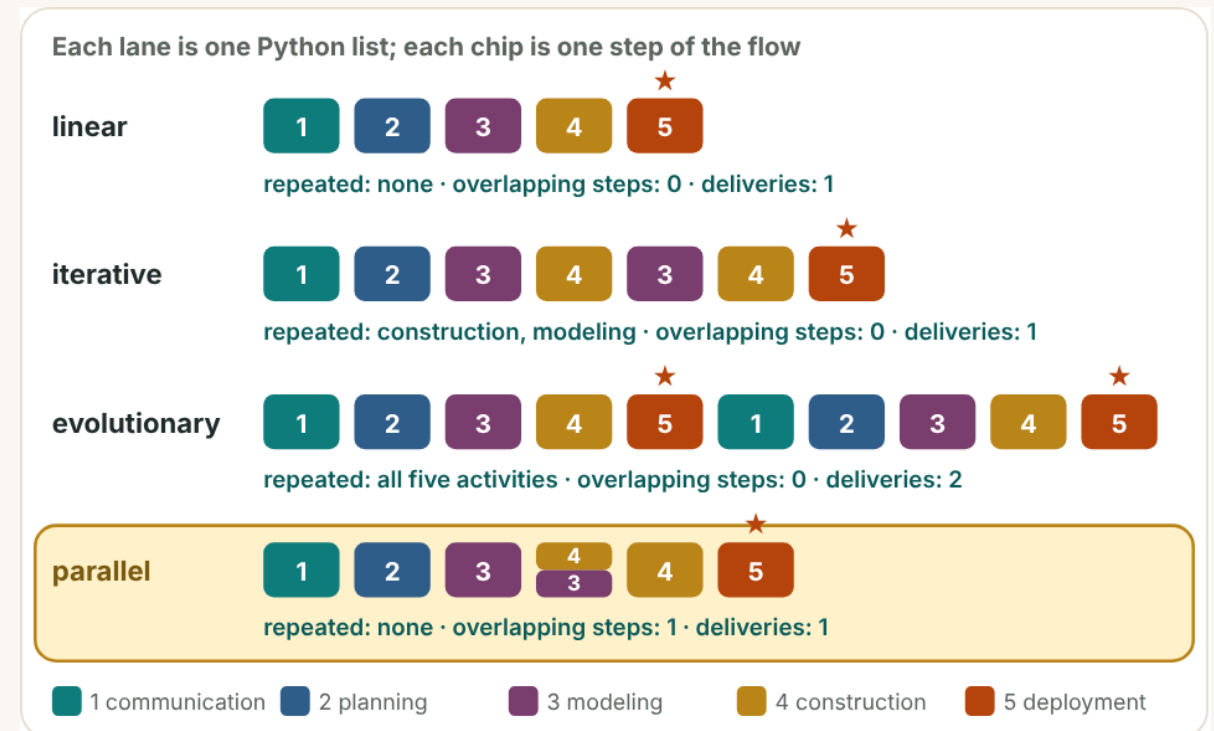
process pattern

A description of a process-related problem met during software work, of the environment in which the problem occurs, and of one or more proven solutions to it.



Real-life example — QueueLess

- QueueLess, first release: the same five activities in four different flows
- Task: Team Falcon plans the first release of QueueLess — a student orders a sandwich and receives a pick-up number.
- Demo 1 (demo1_queueless_process_flows.py) runs it step by step, with a trace and this visualization.
- Comes back in: Block 3; Block 4; Block 10



👁 Visualize-it — Draw your product's process flow

- ▶ On paper, two minutes, alone: draw the five framework activities of your product as five boxes, then connect them the way your team will really work — one straight line (linear), a loop back over some boxes (iterative), a circle (evolutionary) or two tracks side by side (parallel).
- ▶ Under the drawing, write one umbrella activity and the name of the teammate who owns it.
- ▶ Two minutes with a neighbour from another team: find one difference between your two drawings and explain it in one sentence.



What a standard is, and who writes them

- **standard**: consensus document, approved by a recognised body, for common and repeated use
- **IEEE** (Institute of Electrical and Electronics Engineers) — Standards Association
- **ISO** (International Organization for Standardization) and **IEC** (International Electrotechnical Commission) — international bodies
- The three publish the software life-cycle standard together

KEY TERM

standard

A document established by consensus and approved by a recognised body that provides, for common and repeated use, rules, guidelines or characteristics for activities or their results.



The standard in one paragraph

- *Developing a Software Project Life Cycle Process* (2006)
- Directed at the **process architect** of a project
- 1997 edition: 17 activity groups in 5 groups (Annex A)
- Status: superseded — check the status before you cite
- Today: **ISO/IEC/IEEE 12207:2026** (joint standard 12207 of the International Organization for Standardization, the International Electrotechnical Commission and the Institute of Electrical and Electronics Engineers: software life cycle processes)

KEY TERM

process architect

The person who designs a project's life-cycle process: which activities the project needs, in which order, with which methods, templates and tools.



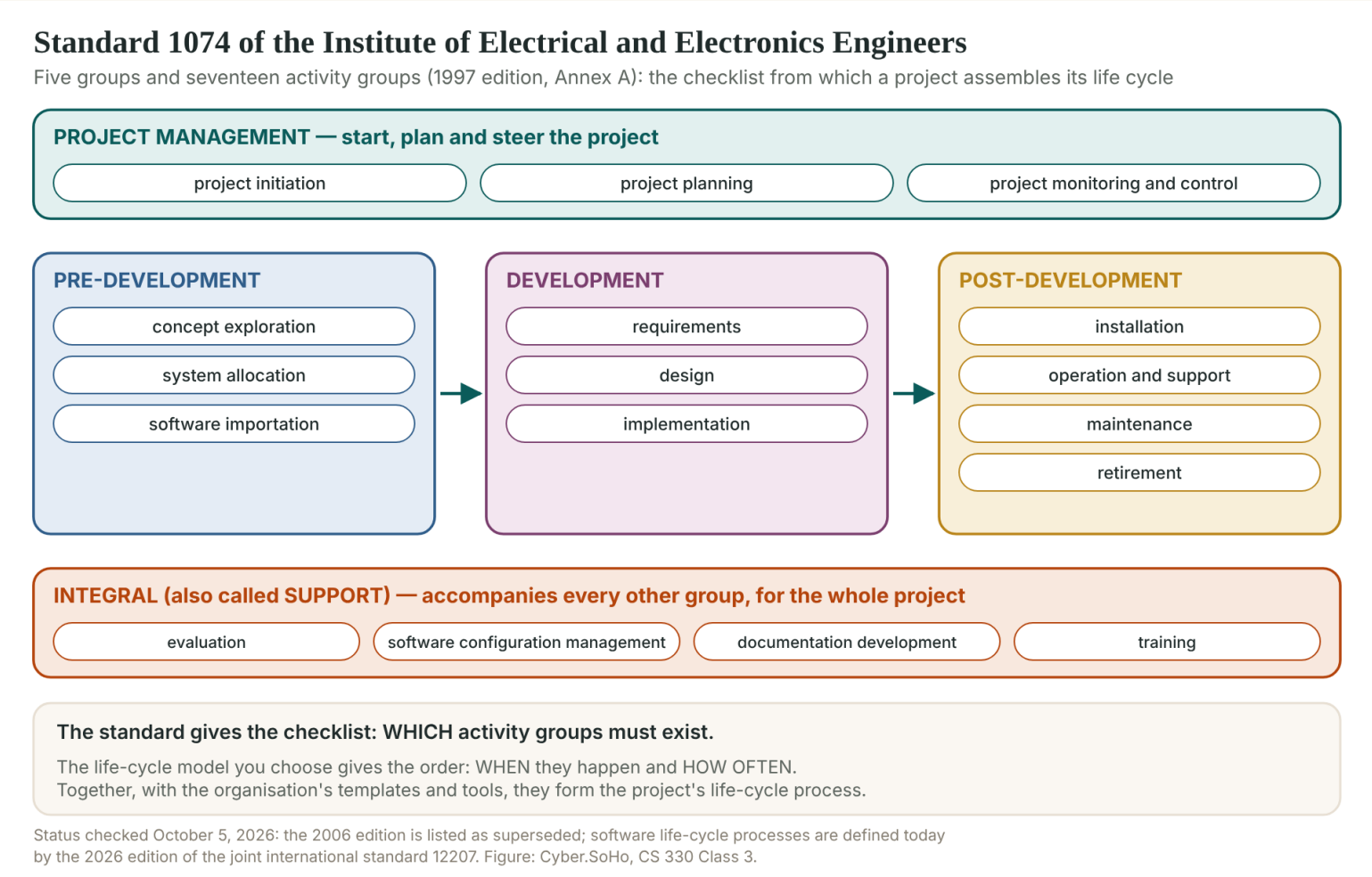
The five groups and their activity groups

Group	Activity groups (1997 edition, Annex A)
Project management	project initiation · project planning · project monitoring and control
Pre-development	concept exploration · system allocation · software importation
Development	requirements · design · implementation
Post-development	installation · operation and support · maintenance · retirement
Integral (support)	evaluation · software configuration management · documentation development · training

- The standard gives the checklist; the model gives the order.



Figure 2 — The five groups of IEEE Std 1074 (Institute of Electrical and Electronics Engineers Standard 1074)



The checklist of IEEE Std 1074 (Institute of Electrical and Electronics Engineers Standard 1074) (1997 edition, Annex A): seventeen activity groups in five groups. The Life-Cycle Map of Block 3 places these groups on your semester.



Integral activities and umbrella activities

- Integral group = the standard's umbrella activities
- evaluation ↔ technical reviews, testing, **SQA** (Software Quality Assurance)
- configuration management ↔ configuration management
- documentation development ↔ work product preparation
- Also called **support** activities

KEY IDEA

Some work belongs to no single phase — evaluation, configuration management, documentation, training — so it must be planned for the whole project. The standard calls it the **integral** group; Lesson 1 called it **umbrella activities**.



The standard gives the checklist; the model gives the order

- A standard says WHAT must exist
- A life-cycle model says WHEN and HOW OFTEN
- Checklist + model + templates = the project's **SPLCP** (Software Project Life Cycle Process)
- This course: Block 1 initiation ... Block 13 installation and maintenance
- Missing everywhere: retirement — what happens to your users' data?

KEY TERM

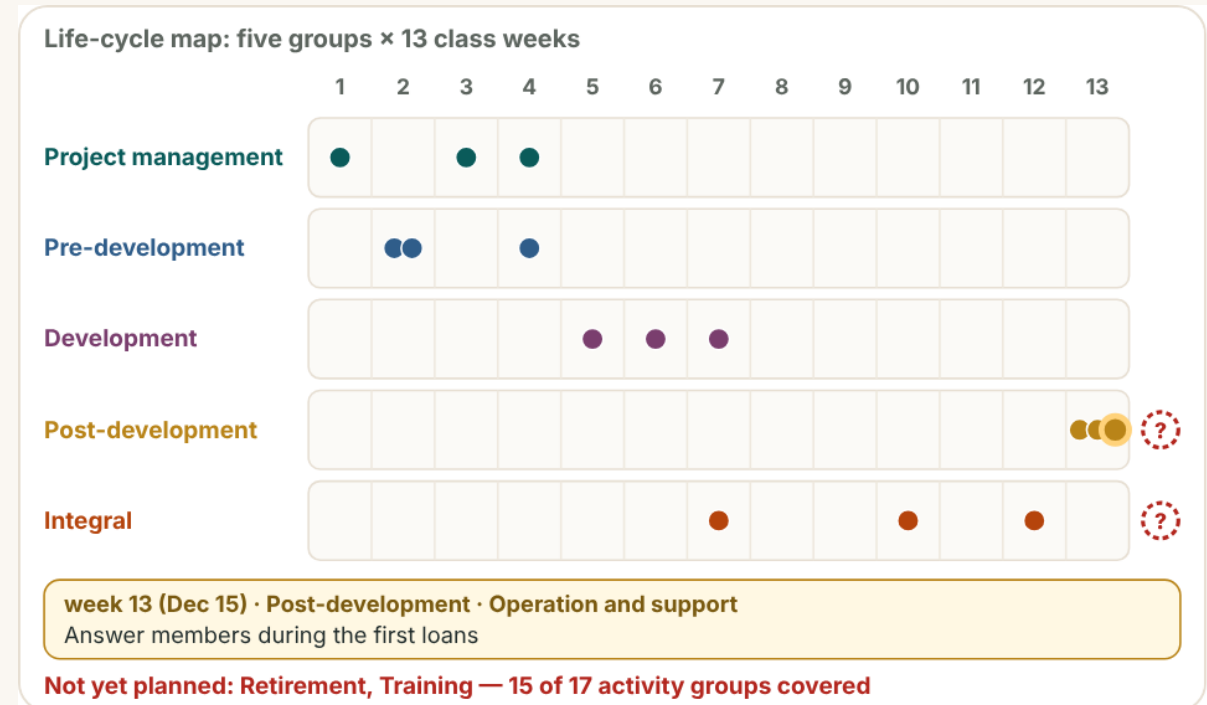
software project life cycle process

The process one project actually follows: the activities chosen from a standard, placed in the order of the chosen life-cycle model, with the organisation's methods, templates and tools attached. **IEEE Std 1074** (Institute of Electrical and Electronics Engineers Standard 1074) abbreviates it **SPLCP** (Software Project Life Cycle Process).



Real-life example — ClubLend

- ClubLend's first life-cycle map: from a to-do list to the five **IEEE** (Institute of Electrical and Electronics Engineers) 1074 groups
- Task: Team Owl (fictional) is building ClubLend, the lending app of a student club that lends board games and books: catalogue, loans, returns, overdue reminders.
- Demo 2 (demo2_clublend_ieee1074_map.py) runs it step by step, with a trace and this visualization.
- Comes back in: Block 3; Block 10; Block 12; Block 13



Flash-cards — Sort the seventeen activity groups into the five groups

ACTIVE BREAK · 0:24–0:28 · 4 MINUTES

- ▶ In pairs, with one deck: lay the five header cards (project management, pre-development, development, post-development, integral) in a row.
- ▶ Two minutes: place the seventeen activity-group cards under the right header.
- ▶ Two minutes: one partner reads an activity group at random, the other names its group without looking; then swap. No deck? Use the five flash-cards of Lesson 2 in this module.



What 'prescriptive' means

- **prescriptive process model:** prescribes process elements + a workflow
- Elements: activities, actions, tasks, work products, quality and change control
- Six models · for each: what it is · when it fits · what it costs

KEY TERM

prescriptive process model

A process model that prescribes a set of process elements — framework activities, actions, tasks, work products, quality-assurance and change-control mechanisms — and a workflow, the way these elements relate to one another.



Waterfall

- **waterfall model** (classic life cycle): one pass, in sequence
- Each phase approved before the next starts
- Fits: stable, well-understood requirements
- Costs: late working version · late error discovery · **blocking states**

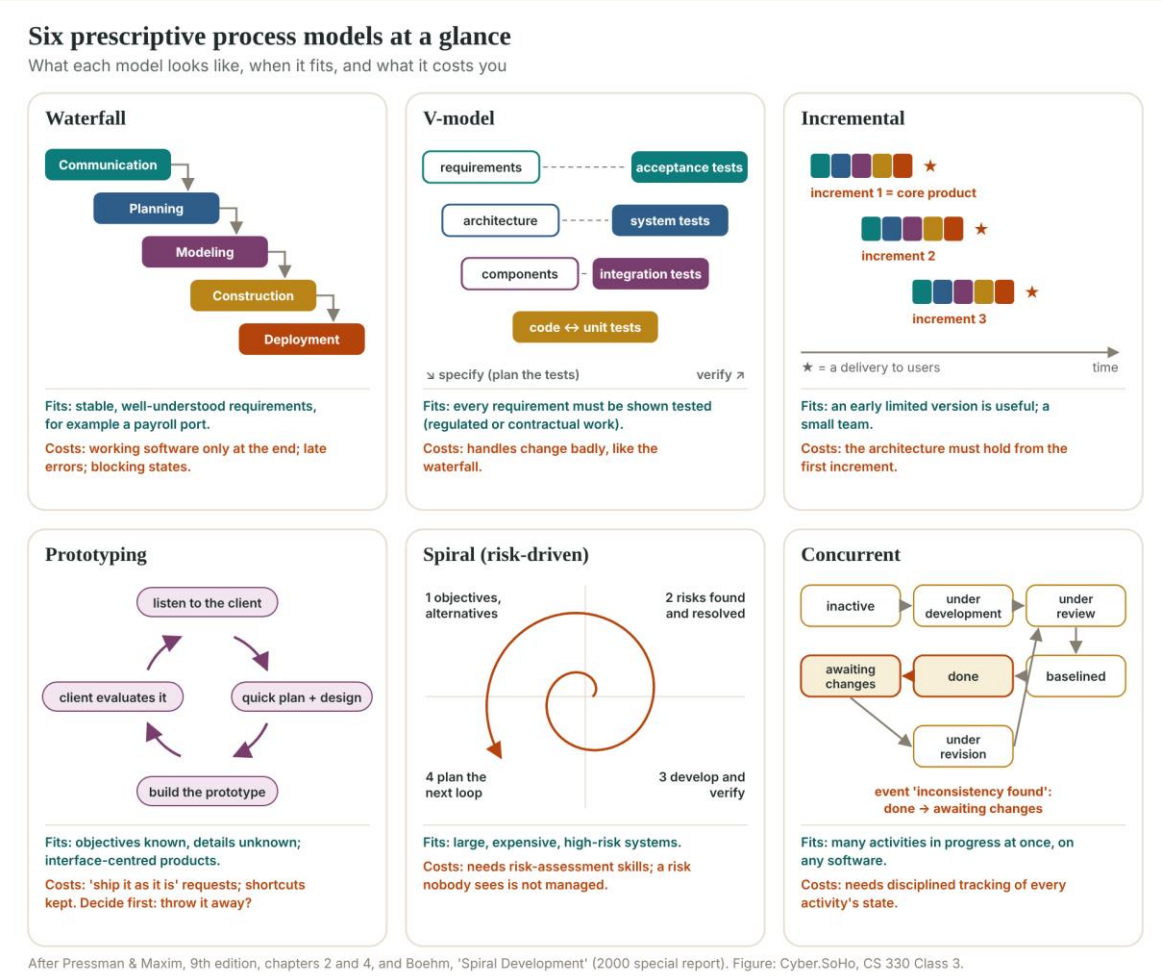
KEY TERM

waterfall model

A linear, sequential model, also called the classic life cycle: communication, planning, modeling, construction and deployment are performed one after the other, and each phase is completed and approved before the next one begins.



Figure 3 — Six prescriptive models at a glance



One panel per model: the shape of the model, when it fits, and what it costs. These six drawings are the ones the professor draws on the board.



V-model

- **V-model:** every development step paired with a test level
- requirements $\leftrightarrow$ acceptance testing
- architectural design $\leftrightarrow$ system testing
- component design $\leftrightarrow$ integration testing
- code $\leftrightarrow$ unit testing · tests *planned* on the way down
- SAAQclic, January 2023: $\approx$ 20 % of final integrated tests not done at go-live confirmation

KEY TERM

V-model

A variation of the waterfall model that pairs every development step with a test level. Going down the left side of the V, the team writes the requirements, the architectural design, the component design and the code; going up the right side, it runs unit, integration, system and acceptance tests. Each test level verifies the step at the same height on the left.



Incremental

- **incremental model:** a series of increments
- Increment 1 = the core product (basic requirements)
- Each next increment adds features, planned with feedback
- Fits: early limited version useful · small team
- Costs: the architecture must hold from increment 1

KEY TERM

incremental model

A model that delivers the software as a series of increments. The first increment is the core product, which satisfies the basic requirements; each following increment adds features, planned with the users' feedback on the previous one. Each increment is produced by a linear sequence, and the sequences are staggered in time.



Prototyping

- **prototyping**: build a partial version quickly to learn the requirements
- **throwaway prototype vs evolutionary prototype**
- Fits: objectives known, details unknown; interface-centred products
- Risks: 'ship it as it is' · shortcuts kept
- Decide BEFORE building: will it be thrown away?

KEY TERM

prototyping

A model in which the team quickly builds a partial version of the software, a prototype, to clarify the requirements with the stakeholders, evaluates it with them and refines it. A **throwaway prototype** is discarded once it has answered its questions; an **evolutionary prototype** is refined into the product.



Spiral

- **spiral model** (Boehm): risk-driven loops
- Each loop: objectives → risks resolved (often a prototype) → develop and verify → plan the next loop
- Early loops: concept, prototype · later loops: fuller versions
- Fits: large, expensive, high-risk systems
- Needs people who can assess risk

KEY TERM

spiral model

A risk-driven process model proposed by Barry Boehm. Development proceeds in loops. In each loop the team determines objectives, alternatives and constraints; evaluates the alternatives and identifies and resolves the risks, often with a prototype; develops and verifies the next version of the product; and plans the next loop.



Concurrent

- **concurrent model:** each activity is a set of states
- inactive · under development · awaiting changes · under review · under revision · baselined · done
- Events move activities between states
- All activities exist at once — the picture a Kanban board gives

KEY TERM

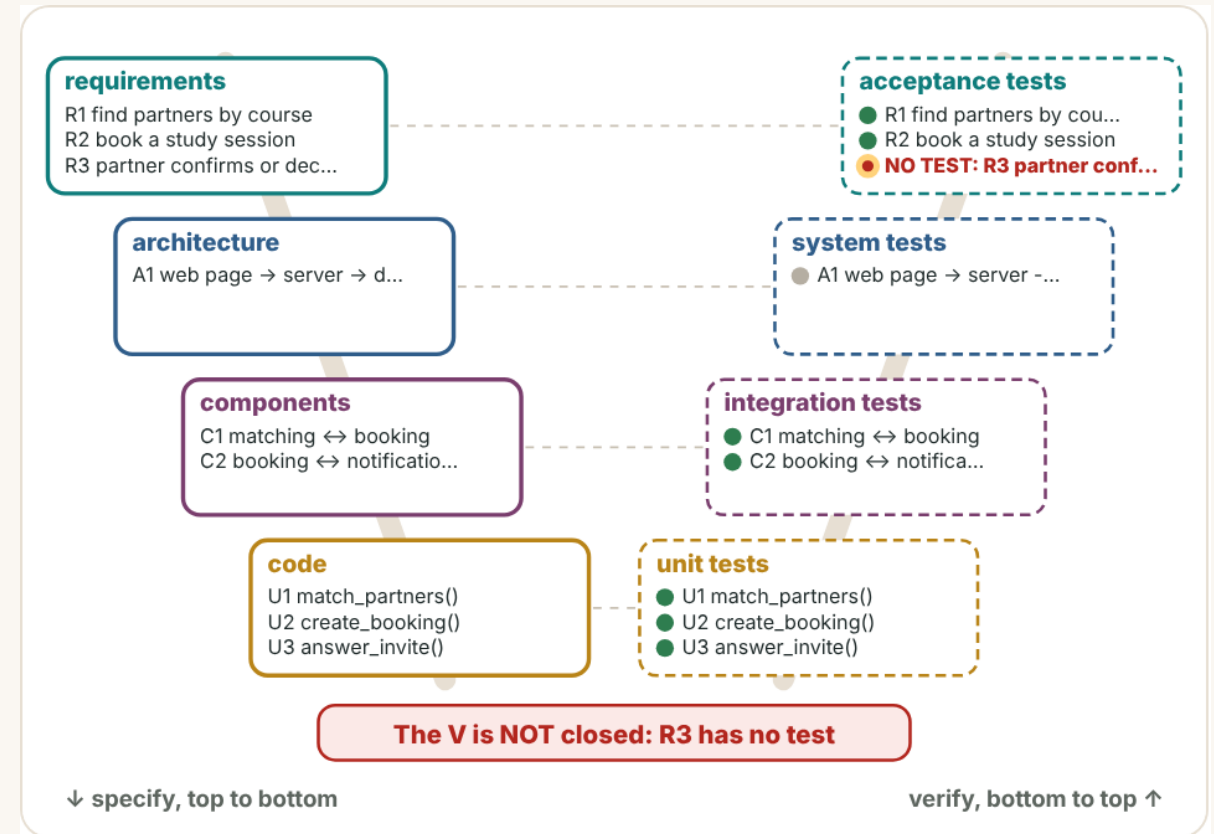
concurrent model

A model that represents every software engineering activity as a set of states — for example inactive, under development, awaiting changes, under review, under revision, baselined, done — together with the events that move an activity from one state to another. All activities exist at the same time, each in its own state.



Real-life example — StudyBuddy

- StudyBuddy's V: planning every test while writing the left side
- Task: Team Lynx (fictional) is building StudyBuddy: find study partners by course and book a study session, which the partner confirms or declines.
- Demo 3 (demo3_studybuddy_v_model.py) runs it step by step, with a trace and this visualization.
- Comes back in: Block 5; Block 7; Block 12



The Unified Process

- **UP** (Unified Process): use-case driven · architecture-centric · iterative and incremental
- Jacobson, Booch, Rumbaugh (1999) · **RUP** (Rational Unified Process) = commercial version
- **Inception** → **Elaboration** → **Construction** → **Transition**
- Iterations inside phases · disciplines in changing proportions
- Elaboration ends with an executable **architecture baseline**

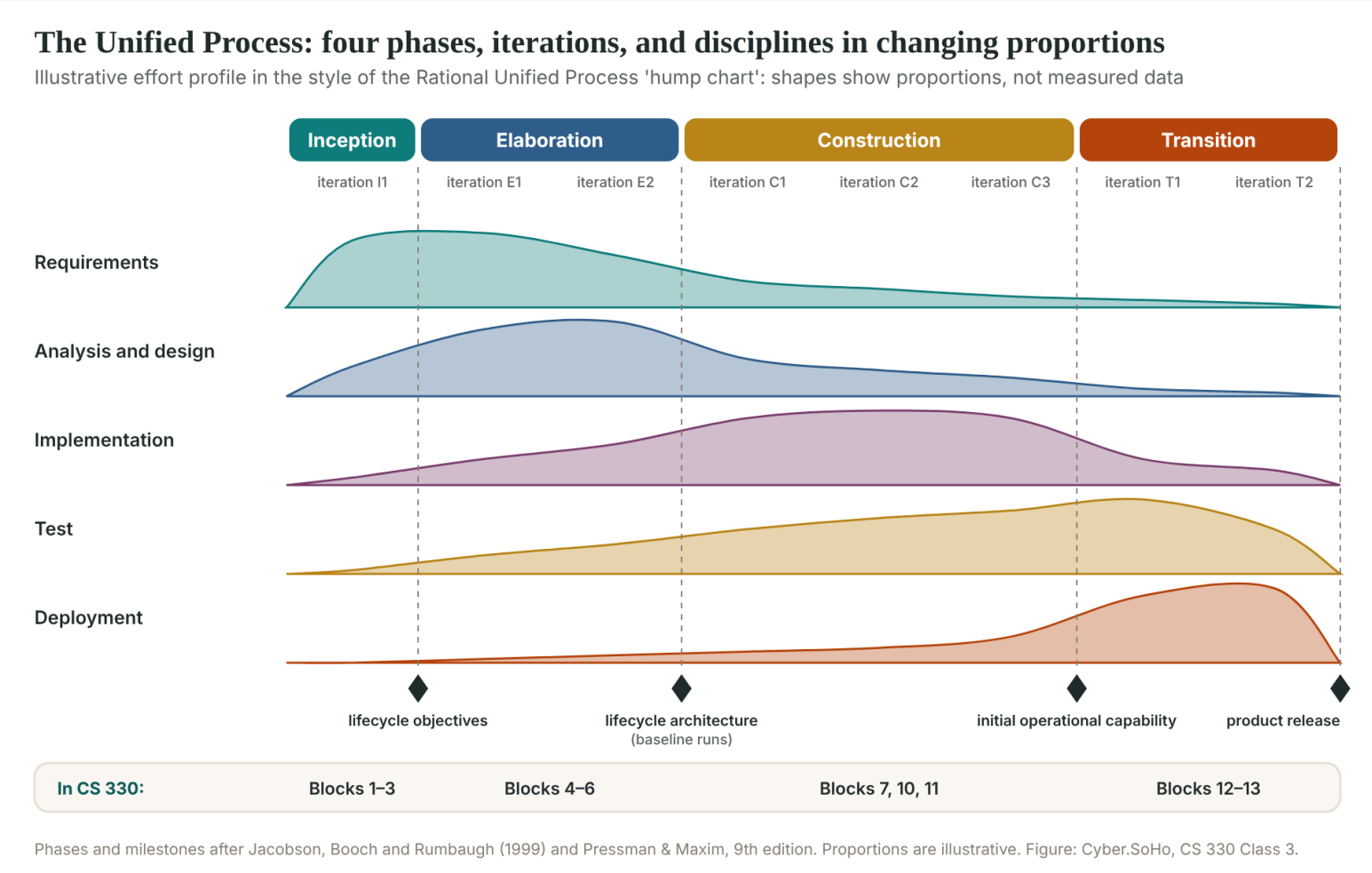
KEY TERM

Unified Process

A use-case driven, architecture-centric, iterative and incremental software process, described in 1999 by Ivar Jacobson, Grady Booch and James Rumbaugh, the authors of the **UML** (Unified Modeling Language). Its best-known commercial version, the **RUP** (Rational Unified Process), came from Rational Software, which **IBM** (International Business Machines) acquired in 2003.



Figure 4 – The Unified Process: phases, iterations, disciplines



Each discipline is present in every phase, in a proportion that changes: requirements early, implementation in construction, deployment in transition. The bottom strip places the phases on the CS 330 Blocks.



Component-based development

- **CBD** (Component-Based Development): build from prepackaged components with defined interfaces
- 1 research and evaluate components · 2 consider integration
- 3 design an architecture around them · 4 integrate · 5 test
- New duties: licence · security updates · imposed design
- The systematic form of the **software importation** activity (Lesson 2)

KEY TERM

component-based development

A process that builds applications from prepackaged software components, commercial or open-source, with well-defined interfaces; abbreviated **CBD** (Component-Based Development).



Formal methods

- **formal methods**: mathematical specification + proof or mechanical checking
- **model checker**: explores every reachable state of a model
- Gains: no ambiguity · finds design errors tests miss
- Costs: time · training · hard for clients to read
- Used at **NASA** (National Aeronautics and Space Administration) · metros (B method) · **AWS** (Amazon Web Services) (**TLA+** (Temporal Logic of Actions), since 2011)

KEY TERM

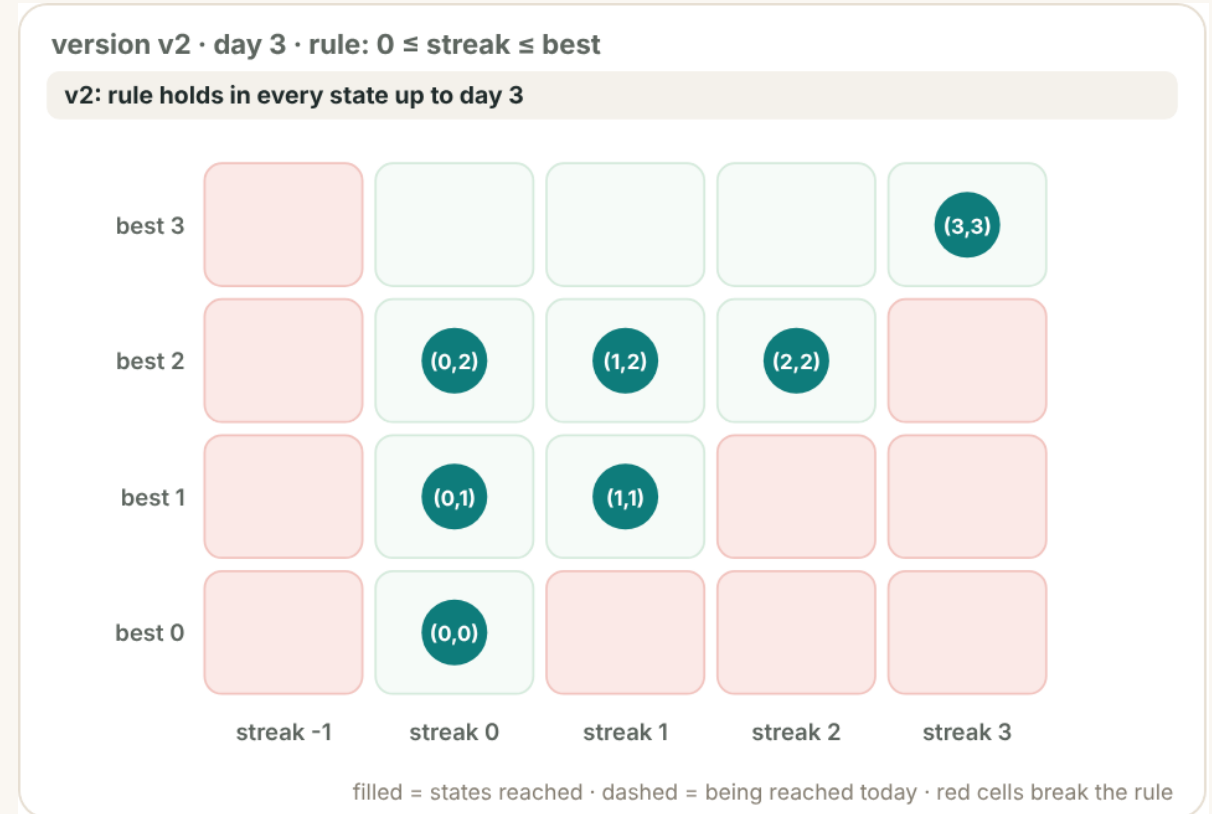
formal methods

Mathematically based techniques for specifying, developing and verifying software. The specification is written in a formal language — logic, set theory — and its properties are proved, or checked mechanically by a **model checker**, a program that explores every reachable state of a model.



Real-life example — HabitHub

- HabitHub: four phases on the course calendar, one reused component, one rule proved on every state
- Task: Team Wren (fictional) is building HabitHub: track daily habits and streaks, with a weekly report.
- Demo 4 (demo4_habithub_streak_check.py) runs it step by step, with a trace and this visualization.
- Comes back in: Block 4; Block 6; Block 7





Fun-quiz — Which model would you pick? Vote with your fingers

ACTIVE BREAK · 0:42–0:46 · 4 MINUTES

- ▶ One hand: 1 finger = waterfall · 2 = V-model · 3 = incremental · 4 = prototyping · 5 = spiral. Listen to the case, count to three, raise your hand.
- ▶ Case 1 — A bank moves its payroll calculation, unchanged, from an old mainframe to a new server. The rules are fixed by law and will not change during the project.
- ▶ Case 2 — A start-up wants a campus food-truck ordering app. The founder 'will know the right screen when she sees it', and the screens are the whole product.
- ▶ Case 3 — A provincial agency replaces a large licensing system used by millions of people. The technology is new to the team, and a failure would be public and expensive.



Fun-quiz — the answers

- ▶ Case 1: waterfall or V-model — stable requirements fixed by law.
- ▶ Case 2: prototyping — unclear interface requirements and an available client.
- ▶ Case 3: spiral — large and high-risk; incremental delivery with a risk review in each increment is also defensible.



Life cycle and process model are not the same thing

- **software life cycle:** conception → retirement (ISO/IEC/IEEE 24765 (joint vocabulary standard 24765 of the International Organization for Standardization, the International Electrotechnical Commission and the Institute of Electrical and Electronics Engineers))
- **SDLC** (Software Development Life Cycle) phases: planning · requirements · design · implementation · testing · deployment · operation and maintenance · retirement
- **process model:** how the team orders, repeats and overlaps that work
- Same life cycle, any model

KEY TERMS

software life cycle — the evolution of a software product from its conception to its retirement.

process model — the strategy that decides how the life-cycle work is carried out: in which order, how many times, with what overlap and with which checkpoints.



Five questions that choose a model

Question	If the answer is ...	... these models fit better
Requirement stability	stable, well understood / unclear, changing	waterfall, V-model / prototyping, incremental, spiral, Unified Process
Risk	high technical or business risk / must prove every requirement	spiral / V-model; formal methods for critical parts
Size	small / large, demanding architecture	incremental, prototyping / Unified Process, spiral
Team	experienced in domain and technology / learning	waterfall or V-model possible / prototyping, incremental
Client availability	often / only at the start and the end	prototyping, incremental, spiral / waterfall, V-model

- Combine when it helps · write the decision down: **ADR** (Architecture Decision Record) — context · ≥ 3 options · decision · consequences



Numbers corner: Basic COCOMO (Constructive Cost Model)

$$E = a \times KLOC^b$$
$$D = c \times E^d$$
$$\text{People} = E \div D$$

effort, in person-months

development time, in months

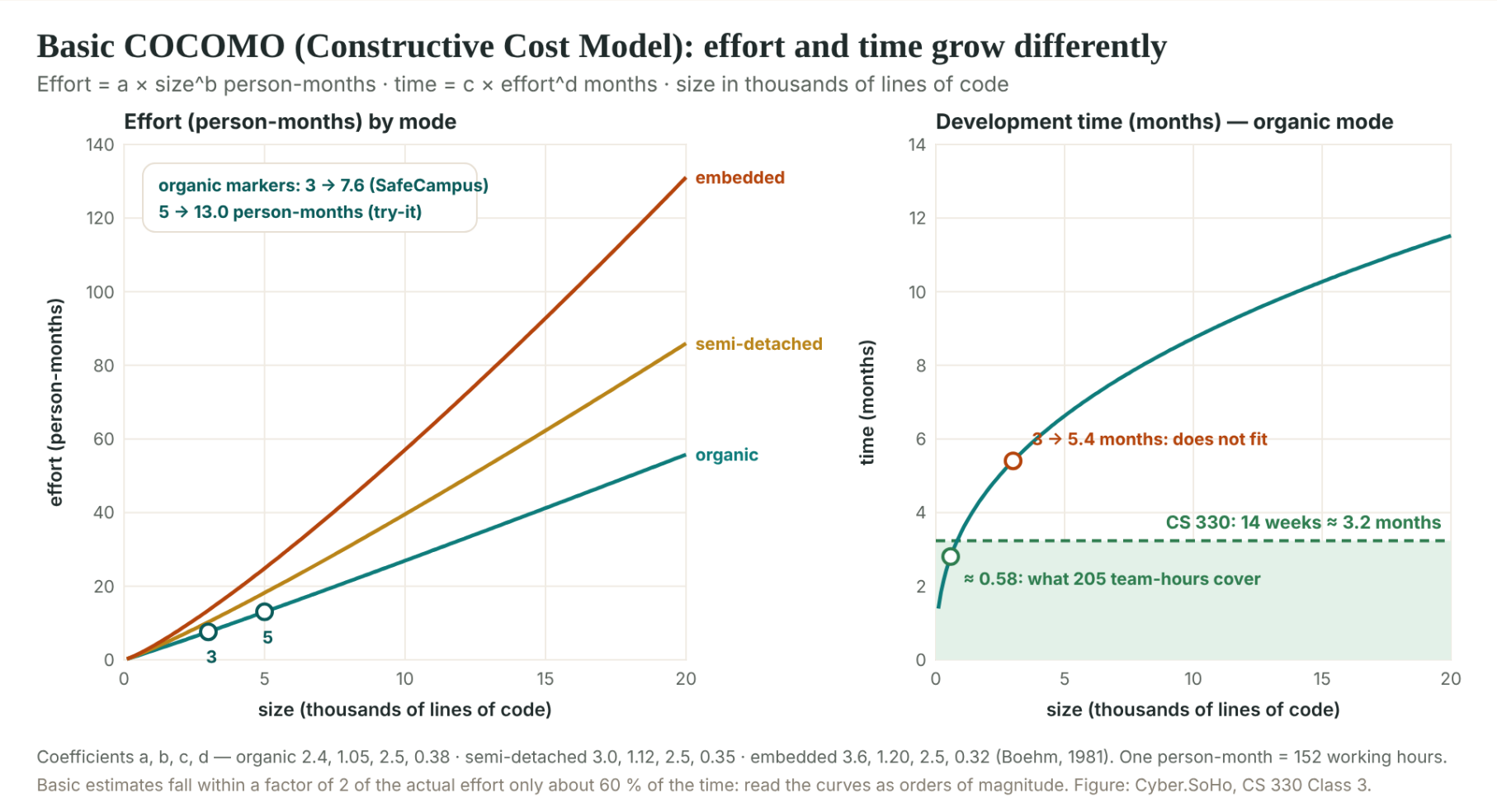
average number of people in parallel

Mode	When it applies	a	b	c	d
organic	small team, familiar application, flexible requirements	2.4	1.05	2.5	0.38
semi-detached	medium team, mixed experience	3.0	1.12	2.5	0.35
embedded	tight hardware, regulatory or operational constraints	3.6	1.20	2.5	0.32

- **KLOC** (thousands of Lines Of Code) = estimated size · one person-month = 152 working hours (**COCOMO** (Constructive Cost Model) II manual)
- Boehm (1981): data from 63 projects · Basic estimates within a factor of 2 of actual only about 60 % of the time



Figure 5 — Basic COCOMO (Constructive Cost Model): effort and time



Effort grows faster than size ($b > 1$); time grows much more slowly than effort ($d < 1$). The green band is what the course calendar allows; the curves come from the same formulas as the demo and the workbook.



The gap sentence

- Course: 14 weeks $\approx$ 3.2 months
- Team of 4 $\times$ 220 min/week $\times$ 14 weeks $\approx$ 205 h $\approx$ 1.35 **PM** (person-month)
- **COCOMO** (Constructive Cost Model): 7.6 **PM** (person-month) over 5.4 months $\rightarrow$ gap $> \times 5$
- Inverted: 1.35 **PM** (person-month) $\approx$ 580 lines of code
- Block 3: one sentence — the numbers + your conclusion

KEY IDEA

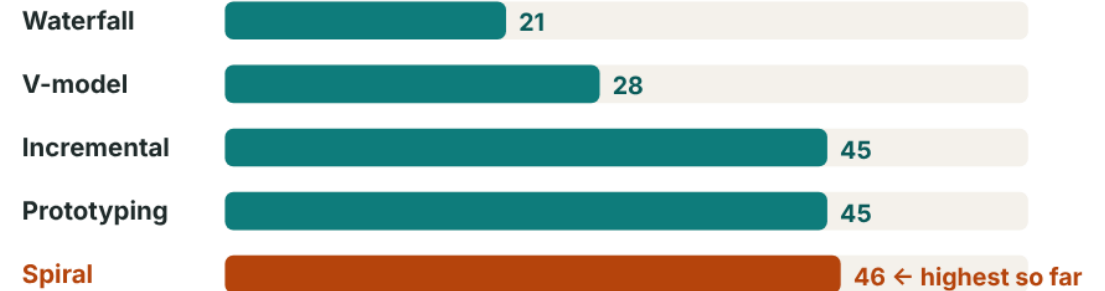
Block 3 needs one sentence about the gap, with the numbers and a conclusion. Two honest conclusions: cut the scope to the stories that fit, or show that a model calibrated on professional teams of the 1970s does not describe a part-time student team — and collect evidence of your real pace. Ignoring the gap is not engineering.



Real-life example — SafeCampus

- SafeCampus: choosing a model with a weighted matrix, then checking the plan with Basic **COCOMO** (Constructive Cost Model)
- Task: Team Heron (fictional) is building SafeCampus: report a hazard or an incident, optionally anonymously, and follow its status.
- Demo 5 (demo5_safecampus_choose_and_estimate.py) runs it step by step, with a trace and this visualization.
- Comes back in: Block 3; Block 4; Mid-Term, Nov 17

Weighted totals (weights × fit, five criteria)



Basic COCOMO (Constructive Cost Model) · organic mode · 3 thousand lines

$E = 7.6$ person-months · $D = 5.4$ months · people ≈ 1.4



With the team's hours, ≈ 578 lines of code fit (Block 3 gap sentence)



Zen-break, then Try-it — Two minutes of quiet, then Basic COCOMO (Constructive Cost Model) for 5 KLOC (thousands of Lines Of Code)

ACTIVE BREAK · 0:56–1:00 · 4 MINUTES

- ▶ Zen-break, two minutes: stand up, roll your shoulders, then breathe in for four counts and out for six, in silence, for one minute. Nothing to do — on purpose.
- ▶ Try-it, two minutes: with the formulas on the screen, compute E, D and the number of people for a product of 5 **KLOC** (thousands of Lines Of Code) in organic mode. Write the intermediate numbers, as on the board.



Zen-break, then Try-it — the answers

- ▶ $E = 2.4 \times 5^{1.05} = 2.4 \times 5.42 \approx 13.0$ person-months
- ▶ $D = 2.5 \times 13.0^{0.38} = 2.5 \times 2.65 \approx 6.6$ months
- ▶ $\text{people} \approx 13.0 \div 6.6 \approx 2$



Studio (100', with your team) — summary

Time	Step	What happens
1:10–1:40	The Model Fair — your LO1 (Learning Outcome 1) debate	Each team draws one model by lottery (waterfall, V-model, incremental, prototyping, spiral, Unified Process), prepares for ten minutes, pitches for three minutes why its model is the best for a campus app, and answers a one-minute rebuttal from another team. The class votes.
1:40–2:20	Process model decision record	For your own product, in the ADR (Architecture Decision Record) format: context, at least three options considered, the decision, and its consequences — what you gain and what you give up.
2:20–2:40	Life-Cycle Map	Your project on a timeline from September 22 to December 20, with the phases, the five IEEE (Institute of Electrical and Electronics Engineers) 1074 groups and every Block placed on it.
2:40–2:50	Numbers	Estimate your product's size in KLOC (thousands of Lines Of Code), compute E and D with Basic COCOMO (Constructive Cost Model), compare with the 14 weeks you actually have, and write one sentence about the gap.
2:50–3:00	Wrap-up	The Block 3 box, the due date, and the exit ticket.



Block 3 — decision record, life-cycle map, estimate

- File: /docs/03-process.md · tag block3
- Process model decision record — context · ≥ 3 options · decision · consequences
- Life-cycle map — Sept 22 → Dec 20, the five **IEEE** (Institute of Electrical and Electronics Engineers) 1074 groups, every Block placed
- Basic **COCOMO** (Constructive Cost Model) estimate + one sentence about the gap with the 14 weeks
- Due: Sunday, October 11, 23:59 · 2 % · link + tag on the **LMS** (Learning Management System)
- Goes to: Mid-Term Part B and Part C; the Final's **LO1** (Learning Outcome 1) debate question



Home Sprint (120') and the terms of the week

AFTER THE HOUR

- Finish Block 3; push the tag block3 before Sunday 23:59
- Read Pressman, 9th edition: chapters 2 and 4; chapter 25, estimation pages
- Numbers practice: Basic **COCOMO** (Constructive Cost Model) for 2, 8 and 20 **KLOC** (thousands of Lines Of Code) — every intermediate number — in /docs/numbers.md
- Flash-cards: process flow, umbrella activity, **IEEE Std 1074** (Institute of Electrical and Electronics Engineers Standard 1074), Waterfall, V-model, incremental model, prototyping, spiral model (risk-driven), Unified Process, **COCOMO** (Constructive Cost Model)
- Optional: one video per lesson; run the five demos and pause where the trace surprises you



Exit ticket

- One sentence, on paper, before you leave:
- “Which model did your team choose, and what is its biggest drawback?”
- Next week — Class 4: agile development and software process improvement ([Agile Manifesto](#))



Videos — all verified, 15 minutes or less (1/2)

WATCH AFTER CLASS

- [Lesson 1 ★ · SE \(Software Engineering\) 2: Generic Process Model | Process Framework Activities with Example — CS & IT Tutorials by Vrushali \(12:40\)](#)
- [Lesson 1 · Software Engineering - 11 - Four Generic Process Models — Access 2 Learn \(6:25\)](#)
- [Lesson 1 · Software Engineering - 05 - The Process Framework — Access 2 Learn \(2:39\)](#)
- [Lesson 2 ★ · ISO \(International Organization for Standardization\) 12207 Systems And Software Engineering – Software Life Cycle Processes — TNV Akademi \(9:30\)](#)
- [Lesson 2 · Understanding the Software Lifecycle Process According to ISO \(International Organization for Standardization\) 12207 — TNV Akademi \(4:39\)](#)
- [Lesson 2 · Fundamental activities of software engineering — Ian Sommerville \(10:24\)](#)
- [Lesson 3 ★ · Software Engineering - 15 - Prescriptive Process Models — Access 2 Learn \(4:27\)](#)
- [Lesson 3 · SE \(Software Engineering\) 12: All SDLC \(Software Development Life Cycle\) Models Revision — CS & IT Tutorials by Vrushali \(12:49\)](#)



Videos — all verified, 15 minutes or less (2/2)

WATCH AFTER CLASS

- [Lesson 3 · Spiral Process - Georgia Tech - Software Development Process — Udacity \(2:31\)](#)
- [Lesson 4 ★ · What is the Rational Unified Process - **RUP** \(Rational Unified Process\)? — Online **PM** \(Project Management\) Courses - Mike Clayton \(12:29\)](#)
- [Lesson 4 · Software Engineering - 16 Specialized Process Models — Access 2 Learn \(3:12\)](#)
- [Lesson 4 · What is Formal Verification? — Galois \(2:29\)](#)
- [Lesson 5 ★ · Basic **COCOMO** \(Constructive Cost Model\) Model | Intro + Example | Example - 1 | Software Engineering — Study with Soumodeep \(8:14\)](#)
- [Lesson 5 · Choosing a Model - Georgia Tech - Software Development Process — Udacity \(2:20\)](#)
- [Lesson 5 · Project Life Cycles Explained: Which One Is Right For You? — Coursera \(6:31\)](#)



Links — official sites (1/4)

- [Cyber.SoHo Legal Notice, Terms of Use and Acceptable Use Policy \(v2.0\)](#)
- [VirusTotal — **URL** \(Uniform Resource Locator, a web address\) scanner](#)
- [VirusTotal — file scanner](#)
- [Google Safe Browsing — site status](#)
- [Python — official site](#)
- [IEEE \(Institute of Electrical and Electronics Engineers\) — Institute of Electrical and Electronics Engineers](#)
- [IEEE \(Institute of Electrical and Electronics Engineers\) Standards Association](#)
- [IEEE Std 1074 \(Institute of Electrical and Electronics Engineers Standard 1074\)-2006 — Developing a Software Project Life Cycle Process](#)
- [ISO \(International Organization for Standardization\) — International Organization for Standardization](#)
- [IEC \(International Electrotechnical Commission\) — International Electrotechnical Commission](#)
- [ISO/IEC/IEEE 12207:2026 \(joint standard 12207 of the International Organization for Standardization, the International Electrotechnical Commission and the Institute of Electrical and Electronics Engineers: software life cycle processes\) — Software life cycle processes](#)
- [ISO/IEC/IEEE 12207:2017 \(joint standard 12207 of the International Organization for Standardization, the International Electrotechnical Commission and the Institute of Electrical and Electronics Engineers: software life cycle processes\) \(withdrawn\)](#)



Links — official sites (2/4)

- [ISO/IEC TR 24774:2010](#) (Technical Report 24774 of the International Organization for Standardization and the International Electrotechnical Commission: guidelines for process description) — Guidelines for process description (withdrawn)
- [ISO/IEC/IEEE 24765:2017](#) (joint vocabulary standard 24765 of the International Organization for Standardization, the International Electrotechnical Commission and the Institute of Electrical and Electronics Engineers) — Systems and software engineering vocabulary
- [IEEE](#) (Institute of Electrical and Electronics Engineers) Computer Society
- [SWEBOK](#) (Software Engineering Body of Knowledge) — Guide to the Software Engineering Body of Knowledge (V4)
- [OMG](#) (Object Management Group) — Unified Modeling Language specification (**UML** (Unified Modeling Language) 2.5.1)
- [Pressman & Maxim, Software Engineering: A Practitioner's Approach, 9th edition](#) (McGraw Hill)
- [SEI](#) (Software Engineering Institute) — Software Engineering Institute, Carnegie Mellon University
- [SEI](#) (Software Engineering Institute) — Spiral Development: Experience, Principles, and Refinements (special report, July 2000)
- [The same spiral report as a PDF](#) (Portable Document Format) (Defense Technical Information Center)
- [CMMI](#) (Capability Maturity Model Integration) Institute (**ISACA** (the professional association formerly called the Information Systems Audit and Control Association))
- [Boehm Center for Systems and Software Engineering](#) — **COCOMO** (Constructive Cost Model) models
- [COCOMO](#) (Constructive Cost Model) 81 — the original model (Basic, Intermediate, Detailed)



Links — official sites (3/4)

- [**COCOMO** \(Constructive Cost Model\) II — the modern successor](#)
- [**COCOMO** \(Constructive Cost Model\) II Model Definition Manual, version 2.1 \(**USC** \(University of Southern California\) Center for Software Engineering\), hosted by the University of Waterloo](#)
- [University of California, Irvine — Basic **COCOMO** \(Constructive Cost Model\) lecture slides](#)
- [University College Cork — **COCOMO** \(Constructive Cost Model\) models \(lecture notes\)](#)
- [University of Waterloo — **COCOMO** \(Constructive Cost Model\) lecture notes \(**SE** \(Software Engineering\) 362\)](#)
- [**IIT** \(Indian Institute of Technology\) Kharagpur Virtual Lab — Project metrics and **COCOMO** \(Constructive Cost Model\)](#)
- [**USC** \(University of Southern California\) Viterbi — tribute to Barry Boehm \(1935–2022\)](#)
- [**NASA** \(National Aeronautics and Space Administration\) Langley Formal Methods group](#)
- [**TLA+** \(Temporal Logic of Actions\) — Leslie Lamport's home page for the language](#)
- [**TLA+** \(Temporal Logic of Actions\) Foundation](#)
- [How Amazon Web Services Uses Formal Methods \(Communications of the **ACM** \(Association for Computing Machinery\), April 2015\)](#)
- [The same article by its **DOI** \(Digital Object Identifier\)](#)



Links — official sites (4/4)

- [ACM \(Association for Computing Machinery\) — Association for Computing Machinery](#)
- [Atelier B \(CLEARSY\) — industrial tool for the B formal method](#)
- [VGQ \(Vérificateur général du Québec, the Auditor General of Québec\) — Vérificateur général du Québec](#)
- [VGQ \(Vérificateur général du Québec, the Auditor General of Québec\), February 2025 report, chapter 2 — the SAAQ \(Société de l'assurance automobile du Québec, the Québec automobile insurance corporation\)'s SAAQclic programme \(in French\)](#)
- [VGQ \(Vérificateur général du Québec, the Auditor General of Québec\) press release of February 20, 2025 \(in French\)](#)
- [VGQ \(Vérificateur général du Québec, the Auditor General of Québec\) chapter 2 as a PDF \(Portable Document Format\) \(in French\)](#)
- [SAAQ \(Société de l'assurance automobile du Québec, the Québec automobile insurance corporation\) — Société de l'assurance automobile du Québec](#)
- [OAG \(Office of the Auditor General of Canada\) — Office of the Auditor General of Canada](#)
- [OAG \(Office of the Auditor General of Canada\), Spring 2018, Report 1 — Building and Implementing the Phoenix Pay System](#)
- [Manifesto for Agile Software Development](#)
- [The twelve principles behind the Agile Manifesto](#)



Legal notice and link safety

- Educational material — © 2026 Cyber.SoHo · All rights reserved
- Licensed to enrolled CS 330 students for personal study — no re-upload, sharing, recording or training of **AI** (Artificial Intelligence) systems
- Terms: [Cyber.SoHo Legal Notice, Terms of Use and Acceptable Use Policy](#)
- Check every link and downloaded file with [VirusTotal](#) (section 14 and Annex C)
- Every team, person and organisation in the examples is fictional (sections 6.7 and 18.2)
- No affiliation with any vendor, platform, standards body or institution named (section 5)

